

**CS6004NI: Application Development
30% Group Coursework
Autumn 2025-26
Credit: 30**

Group Name:			
SN	Student Name	College ID	University ID
	Bipin Kumar Thapa	NP01CP4A230088	23050398
	Dishant Panjiyar	NP01CP4A230308	23050260
	Kushal Chandra Shrestha	NP01CP4A230063	23050337
	Pratik Prasai	NP01CP4A230047	23047619
	Slesha Rawal	NP01CP4A230378	23050305

Assignment Due Date: Wednesday, April 29, 2026

Assignment Submission Date: Wednesday, April 29, 2026

Word Count: 2582

Module Leader: Mr. Bikram Poudel

Project File Links:

Link:	https://github.com/Kushal-C-Shrestha/apd-frontend https://github.com/Kushal-C-Shrestha/apd-backend
--------------	--

I confirm that I understand my coursework needs to be submitted online via My Second Teacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Table of Contents

1	Overview	1
1.1	Purpose.....	1
1.2	Scope.....	2
1.3	Aims and Objectives	2
1.3.1	Aim.....	2
1.3.2	Objectives	2
2	Tech Stack	3
3	Features and functionalities	5
3.1	Completion status	5
3.2	Completed features.....	6
3.2.1	Member 1: Bipin Kumar Thapa	6
3.2.2	Member 2: Dishant Panjiyar.....	10
3.2.3	Feature 5: Admin can manage vendor details (CRUD operations)	17
3.2.4	Frontend:	22
3.2.5	Member 3: Kushal Chandra Shrestha.....	23
3.2.6	Member 4: Pratik Prasai	31
3.2.7	Member 5: Slesha Rawal.....	41
4	Proof of Work	46
5	Individual reflection	47
5.1	Member 1: Bipin Kumar Thapa	47
5.2	Member 2: Dishant Panjiyar.....	47
5.3	Member 3: Kushal Chandra Shrestha.....	47
5.4	Member 4 : Pratik Prasai	48
5.5	Member 5 : Slesha Rawal	48

Table of Figures

Figure 1: Adding new customer with vehicle details from Frontend	6
Figure 2: Backend API for customer registration and Search.....	6
Figure 3: Details required to register	7
Figure 4: Successfully Customer registration.	7
Figure 5: Customer Search Interface	8
Figure 6: Customer Search API Response	9
Figure 7: Parts List Retrieval API (GET /api/Parts)	11
Figure 8: Delete Part Operation (DELETE /api/Parts/{id}	12
Figure 9: Update Part Details (PUT /api/Parts/{id})	13
Figure 10: Add New Vendor	19
Figure 11: Update Vendor Details	20
Figure 12: Delete Vendor	21
Figure 13: Frontend part of Vendor And Parts management.....	22
Figure 14: List of API for managing Appointment.....	23
Figure 15: Create Appointment Request.....	24
Figure 16: Appointment Saved in Database.....	24
Figure 17: Get All Appointments (GET /api/appointment)	25
Figure 18: Fetch User Appointments by searching with userId (GET /api/appointment/user/{userId})	25
Figure 19: Successfully Retrieved Appointment by userId	26
Figure 20: Part request API.....	26
Figure 21: Creates a part request for unavailable parts.	26
Figure 22: Get All Part Requests.....	27
Figure 23: Get User Part Requests	28
Figure 24: List of Api to POST and GET the review	28
Figure 25: Yearly Financial Report.....	31
Figure 26: Api to fetch monthly financial report	33
Figure 27: Financial Report Yearly	34
Figure 28: Regulars Report Response	38
Figure 29: Pending Credits Report Response	39
Figure 30: High spender's Report Response	40

Figure 31: Fetch all the available sales	41
Figure 32: Create Sale Request.....	42
Figure 33: Generated invoice of sales.....	44
Figure 34: Customer Purchase History Retrieval	45
Figure 35: ER Diagram Representing Tables and Their Relationships	46

Table of Tables

Table 1: Feature completion status table.	5
--	---

1 Overview

This project entails the development of a complete web application designed for a vehicle service and parts retail center. The system is intended to streamline and efficiently manage day-to-day operations, including the tracking of inventory, the processing of sales, customer management, and financial record-keeping. With distinct role-based access for admins, staff and customers alike, users are restricted to the tasks for which they are authorized. Essential features of the system include part management, invoice creation, service booking and automated notification generation to improve the efficiency of the operation and minimize manual intervention.

As well as these core features, the system endeavors to offer an intelligent and user-friendly experience. Data is organized in a systematic fashion allowing immediate access to information; staff are able to quickly find part and customer information whereas admins can readily view stock levels and performance reports. The customer too can enjoy many features such as viewing their own service history and being automatically alerted when vehicle maintenance is due.

This project is aimed to mimic a real-world business environment where multiple individuals need to use the same application to complete tasks. Not only are functional requirements met, but also user-friendliness, security and scalability; a practical tool for managing a vehicle service and parts business.

1.1 Purpose

The purpose of this project is to practice and demonstrate the skills I've acquired with C# and ASP.NET Core through the development of a real-world application. In doing so, I will be able to showcase the knowledge I've gained in aspects such as; System Design, Coding practices, database implementation and user interface creation.

In addition to the practical nature of this project, I intend to create efficient and well-written code with appropriate error handling and security considerations; and present it alongside a detailed report to show how I achieved this, the features it implements and why the decisions that have been made have been made.

1.2 Scope

The system will support three user types:

Admin: Manage Staff, Vendors, Vehicle Parts, purchases and produce reports. They will monitor stock and overall business performance.

Staff: Manage everyday operations of the business such as, customer registration, sale processing, stock search and send customers notifications and emails.

Customer: Be able to register / log into the application to book vehicle services and view their service history and purchases history, also to be able to receive vehicle maintenance notifications (AI suggestions possible if included).

All of the required features such as low stock alerts, loyalty discount, invoice generation, and reports will be developed. The application will be implemented using ASP.NET Core with PostgreSQL.

1.3 Aims and Objectives

1.3.1 Aim

The main objective of this project is to create a simple, user-friendly and efficient web application for the real-time handling of vehicle services, parts inventory, sales, and customer communication processes.

1.3.2 Objectives

The objectives of this project is listed below:

- To implement all the main functionalities for parts and vendors CRUD operations, sales operations, invoice generation and customer reports.
- To add extended features including automated e-mail notifications, low-stock alerts, and loyalty discounts system.
- To produce quality code that is structured well, highly readable and incorporates appropriate error handling.

- To develop an easy-to-use interface for the application that is quick and efficient for any user to navigate.
- To create proper documentation of this project.

2 Tech Stack

- Frameworks: For backend of the application, .NET 8.0 is used which as ASP.NET Core Web API serves as main framework to deal with business logic, API requests, authentication, and database manipulation. For the frontend, React is utilized in order to produce responsive user interface.

Backend: .NET 8.0 (ASP.NET Core Web API)

Frontend: React 19 (with Vite)

- External libraries:

Backend: To support database interaction, authentication and API documentation, following libraries have been used.

- Entity Framework Core (v8.0.0): The object-relational mapper used to easily translate the C# object into the corresponding database tables.
- Npgsql.EntityFrameworkCore.PostgreSQL (v8.0.0): PostgreSQL database provider to establish communication between Entity Framework Core and the PostgreSQL database.
- Microsoft.AspNetCore.Authentication.JwtBearer (v8.0.0): Provides mechanism to authenticate users through JSON web token, allowing access only for authorized users to protected API routes.
- Microsoft.AspNetCore.Identity.EntityFrameworkCore (v8.0.0): This gives us basic support for user authentication and management of roles, we are able to perform role based access for Admin, Staff and Customers.

- Swashbuckle.AspNetCore (v6.6.2): To generate the Swagger UI to test API endpoints on the fly during the development phase.

Frontend : Following are the important libraries used for frontend development.

- React (v19.2.5): A JavaScript library which is used to create the User Interface of the application. Using React we can create reusable components which leads to faster, interactive, and dynamic UI.
- React DOM (v19.2.5): Used with React to render React components to the DOM in the browser. It can efficiently update and render the DOM.
- React Router DOM (v7.14.2): Used for managing routes within the application. Single-page application navigation could be handled, like navigating between different views such as login, dashboard, inventory, etc, without refreshing the whole page.
- Axios (v1.15.2): Promise-based HTTP client used to make HTTP requests from the frontend to the backend API. Primarily used to retrieve data, add new data, update and delete data from the server.
- Lucide React (v1.11.0): This is a clean and customizable icon library used for designing User Interface elements.
- Database: PostgreSQL

3 Features and functionalities

3.1 Completion status

S.No	Feature	Assigned to	Completion status
1.	Admin can generate and view financial reports (daily, monthly, yearly)	Pratik	Completed
2.	Admin can manage staff registration and roles	Bipin	Pending
3.	Admin can perform parts management (purchase, edit, delete)	Dishant	Completed
4.	Admin can create purchase invoices for stock updates	Dishant	Pending
5.	Admin can manage vendor details (CRUD operations)	Dishant	Completed
6.	Staff can register new customers with vehicle details	Bipin	Completed
7.	Staff can sell vehicle parts and create sales invoices	Slesha	Completed
8.	Staff can view customer details, history, and vehicle info	Slesha	Completed
9.	Staff can generate customer-related reports (regulars, high spenders, pending credits)	Pratik	Completed
10	Staff can search customers by vehicle number, phone, ID, or name	Bipin	Completed
11	Staff can send invoices via email to customers	Slesha	Pending
12	Customers can self-register and manage profile & vehicle details	Bipin	Pending
13	Customers can book appointments, request unavailable parts, and review services	Kushal	Completed
14	Customers can view their purchase/service history	Pratik	Pending
15	System automatically notifies Admin for low stock (<10) and sends email reminders to customers with unpaid credits for more than 1 month	Kushal	Pending
16	Loyalty Program: Customers get 10% discount if they spend more than 5000 in a single purchase	Kushal	Pending

Table 1: Feature completion status table.

3.2 Completed features

3.2.1 Member 1: Bipin Kumar Thapa

Feature 6: Staff can register new customers with vehicle details

Frontend:

The screenshot shows a web form titled "Register Customer". At the top right, there are two buttons: "Register" and "Search". The form is divided into two main sections: "Customer Information" and "Vehicle Details".

Customer Information (Personal & contact details):

- Full Name *: Bipin Thapa
- Phone Number *: 9822321123
- Email Address *: bipin369thapa@gmail.com
- Address: Lalitpur

Vehicle Details (Primary vehicle registration):

- Vehicle Number *: CAT CHA 3492
- Brand: Hero
- Model: Pleasure
- Year: 2016

At the bottom right of the "Vehicle Details" section, there are two buttons: "Clear Form" and "Register Customer".

Figure 1: Adding new customer with vehicle details from Frontend

Backend:

The screenshot shows the Swagger API interface for "VehicleAPI". At the top, there is a "Select a definition" dropdown menu with "VehicleAPI v1" selected. Below the header, the API is identified as "VehicleAPI 1.0 OAS 3.0" with the URL "https://localhost:7042/swagger/v1/swagger.json".

The "Customer" section is expanded, showing two endpoints:

- POST** /api/Customer/register
- GET** /api/Customer/search

Figure 2: Backend API for customer registration and Search

Customer

POST /api/Customer/register

Parameters

No parameters

Request body

application/json

```
{
  "fullName": "Shyam",
  "email": "Joshi",
  "phone": "9844633775",
  "address": "Balkhu",
  "vehicleNumber": "BA1 CHA 4578",
  "brand": "KTM",
  "model": "dune",
  "year": 2017
}
```

Execute

Responses

Figure 3: Details required to register

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7042/api/Customer/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "fullName": "Shyam",
    "email": "Joshi",
    "phone": "9844633775",
    "address": "Balkhu",
    "vehicleNumber": "BA1 CHA 4578",
    "brand": "KTM",
    "model": "dune",
    "year": 2017
  }'
```

Request URL

https://localhost:7042/api/Customer/register

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "message": "Customer registered successfully.",
  "data": {
    "userId": 20,
    "customerId": "CUS-5440",
    "fullName": "Shyam",
    "email": "Joshi",
    "phone": "9844633775",
    "address": "Balkhu",
    "isActive": true,
    "createdAt": "2026-04-29T06:05:06.1089659Z",
    "vehicleNumber": "BA1 CHA 4578",
    "brand": "KTM",
    "model": "dune",
    "year": 2017
  }
}
```

Figure 4: Successfully Customer registration.

Feature 10: Staff can search customers by vehicle number, phone, ID, or name

Frontend:

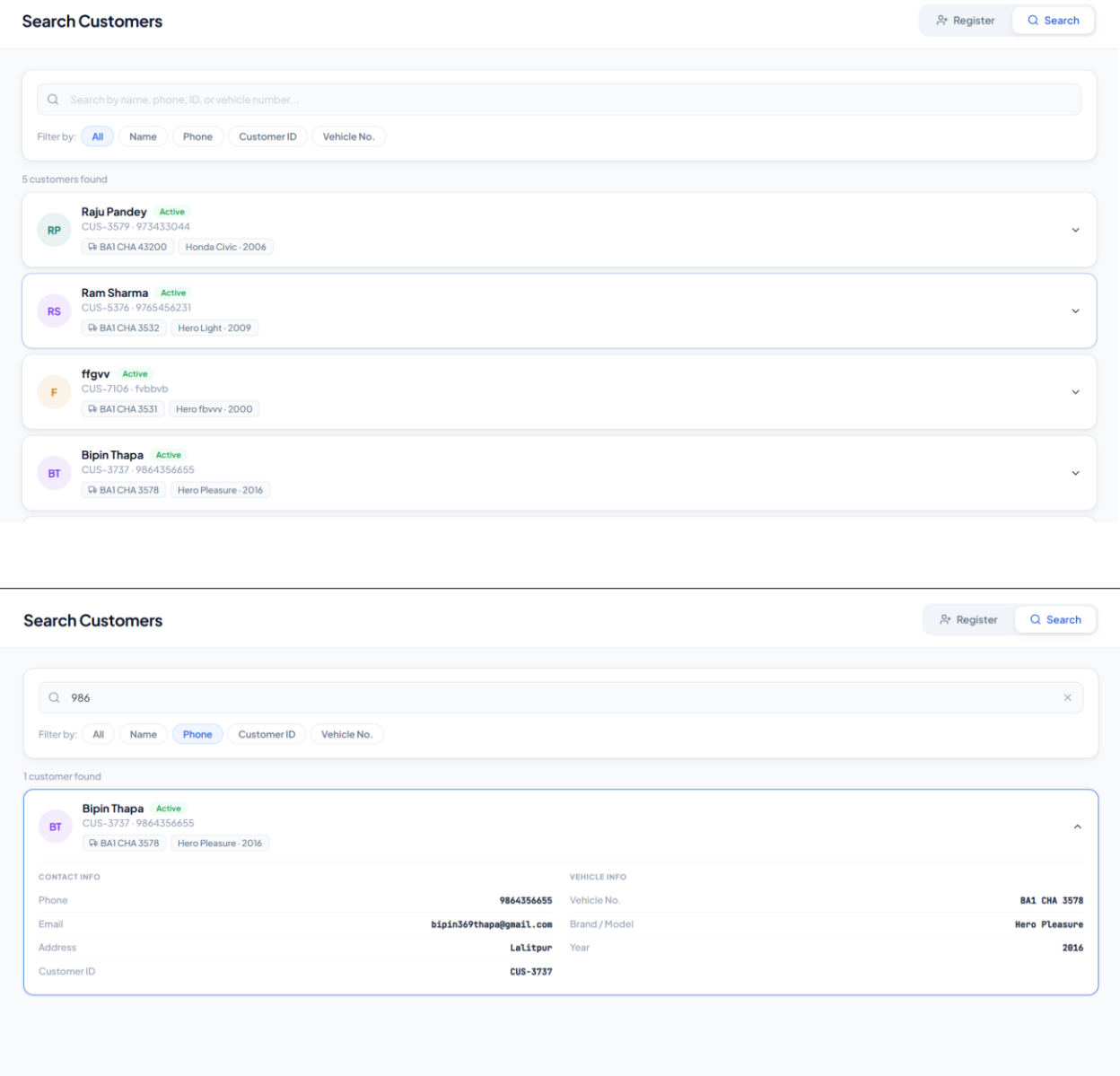


Figure 5: Customer Search Interface

Backend:

The screenshot displays a REST client interface for the `GET /api/Customer/search` endpoint. The parameters section shows a query string `Shyam` and a filter `all`. The response section shows a 200 OK status with a JSON body containing customer details.

Parameters

Name	Description
query	Shyam
filter	all

Responses

Code	Description	Links
200	OK	No links

Curl

```
curl -X 'GET' \
  'https://localhost:7042/api/Customer/search?query=Shyam&filter=all' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:7042/api/Customer/search?query=Shyam&filter=all
```

Server response

Code	Details
200	Response body <pre>{ "success": true, "data": [{ "userId": 20, "customerId": "CUS-5448", "fullName": "Shyam", "email": "Joshi", "phone": "9844633775", "address": "Balkhu", "isActive": true, "createdAt": "2025-04-29T06:05:06.108965Z", "vehicleNumber": "BA1 CHA 4578", "brand": "KTM", "model": "dune", "year": 2017 }] }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 29 Apr 2025 06:10:21 GMT server: Kestrel</pre>

Figure 6: Customer Search API Response

POST `/api/Customer/register` - Registers a new customer along with vehicle details. It automatically generates a customer ID and password after registration and sends them to the customer's email.

GET `/api/Customer/search` - Fetches all registered customers with detailed information. Allows filtering by name, phone, address, and vehicle number.

3.2.2 Member 2: Dishant Panjiyar

Feature 3: Admin can perform parts management (purchase, edit, delete)

Backend:

Parts		^
GET	/api/Parts	▼
POST	/api/Parts	▼
GET	/api/Parts/{id}	▼ 
PUT	/api/Parts/{id}	▼
DELETE	/api/Parts/{id}	▼
GET	/api/Parts/purchases	▼
POST	/api/Parts/purchases	▼
GET	/api/Parts/purchases/{id}	▼

- **GET /api/Parts** - Fetches all parts from the database and returns them as a list. This is what loads Parts in Inventory table.
- **POST /api/Parts** - Creates a new part. Receives name, description, unit price, and stock quantity, saves it to the database, and returns the created part.
- **GET /api/Parts/{id}** - Fetches a single part by its ID. Used internally when needed to provide details of one specific part.
- **PUT /api/Parts/{id}** - Updates an existing part by ID. Receives the updated name, price, stock quantity etc. and overwrites the existing record.
- **DELETE /api/Parts/{id}** - Deletes a part by ID permanently from the database. Returns a conflict error if the part is referenced in purchases.
- **GET /api/Parts/purchases** - Fetches all purchase records with their items and vendor info. This is what loads the Purchase History table.
- **POST /api/Parts/purchases** - Records a new purchase. Receives a vendor ID and list of items (part, quantity, unit cost), saves the purchase, and automatically increases the stock quantity of each part involved.
- **GET /api/Parts/purchases/{id}** - Fetches a single purchase record by ID including all its line items.

GET /api/Parts

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:44310/api/Parts' \
  -H 'accept: */*'
```

Request URL

https://localhost:44310/api/Parts

Server response

Code

Details

200

Response body

```
[
  {
    "partid": 1,
    "name": "bike part",
    "description": "shock",
    "unitPrice": 1000,
    "stockQuantity": 20,
    "createdAt": "2026-04-29T03:08:59.220436Z"
  },
  {
    "partid": 2,
    "name": "string",
    "description": "string",
    "unitPrice": 0.01,
    "stockQuantity": 2147483647,
    "createdAt": "2026-04-29T03:09:40.131417Z"
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 06:19:30 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code

Description

200

OK

Links

No links

Figure 7: Parts List Retrieval API (GET /api/Parts)

11

QueryQuery History

1SELECT * FROM public."Parts"
2ORDER BY "PartId" ASC

Data OutputMessagesNotifications

+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

Showing rows: 1 to 2✎

	PartId [PK] integer ✎	Name character varying (100) ✎	Description text ✎	UnitPrice numeric ✎	StockQuantity integer ✎	CreatedAt timestamp with time zone ✎
1	1	bike part	shock	1000	20	2026-04-29 08:53:59.220436+05:...
2	2	string	string	0.01	2147483647	2026-04-29 08:54:40.131417+05:...

DELETE /api/Parts/{id}

Parameters

Cancel

Name	Description
id * required integer(\$int32) (path)	2

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'https://localhost:44310/api/Parts/2' \
  -H 'accept: */*'

```

Request URL

https://localhost:44310/api/Parts/2

Server response

Code	Details
200	Response body <pre>{ "message": "Part deleted successfully." }</pre> 📄 Download Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 29 Apr 2026 06:24:13 GMT server: Microsoft-IIS/10.0 x-powered-by: ASP.NET</pre>

Responses

Code	Description
200	OK

Link to this response

No links

Figure 8: Delete Part Operation (DELETE /api/Parts/{id})

QueryQuery History

1

SELECT * FROM public."Parts"

2

ORDER BY "PartId" ASC

Data OutputMessagesNotifications

Showing rows: 1 to 1

	PartId [PK] integer	Name character varying (100)	Description text	UnitPrice numeric	StockQuantity integer	CreatedAt timestamp with time zone
1	1	bike part	shock	1000	20	2026-04-29 08:53:59.220436+05:...

PUT/api/Parts/{id}

Parameters

Cancel

Name	Description
id * required integer(\$int32) (path)	id

Request body

application/json

{

"name": "string",

"description": "string",

"unitPrice": 0.01,

"stockQuantity": 2147483647

}

Execute

Responses

Code	Description	Links
200	OK	Activat No links Go to Set

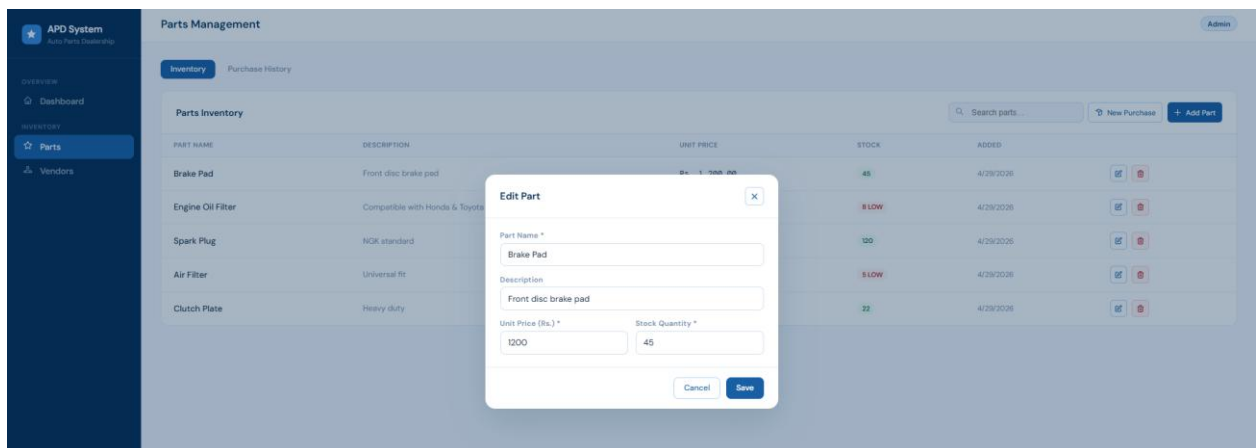
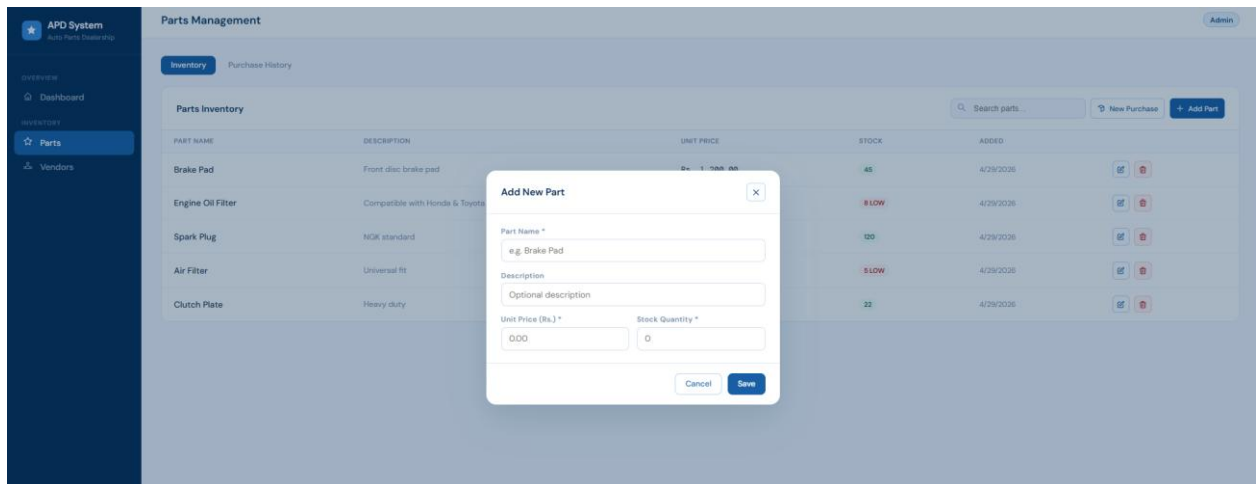
Figure 9: Update Part Details (PUT /api/Parts/{id})

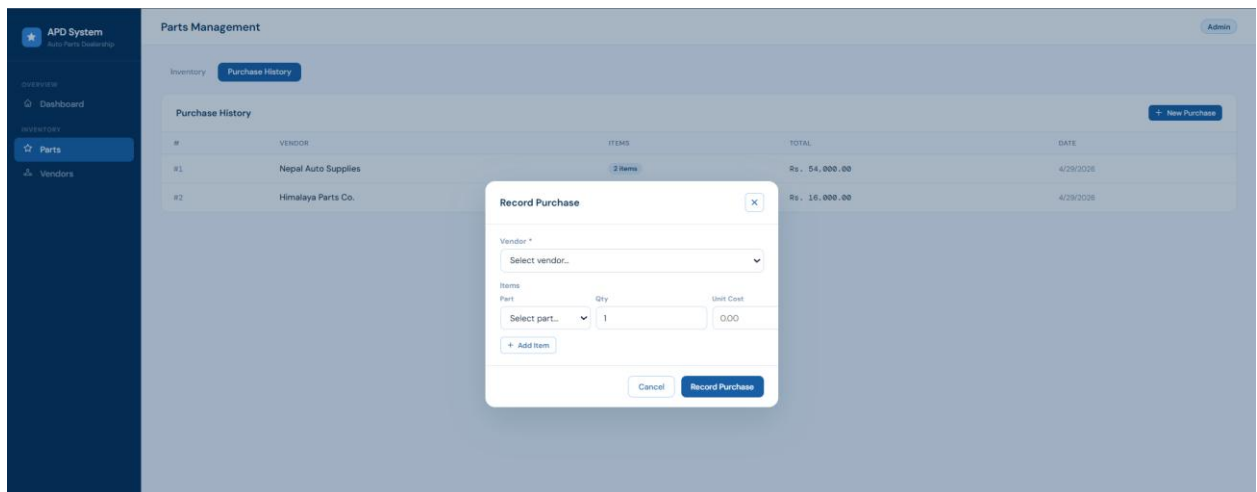
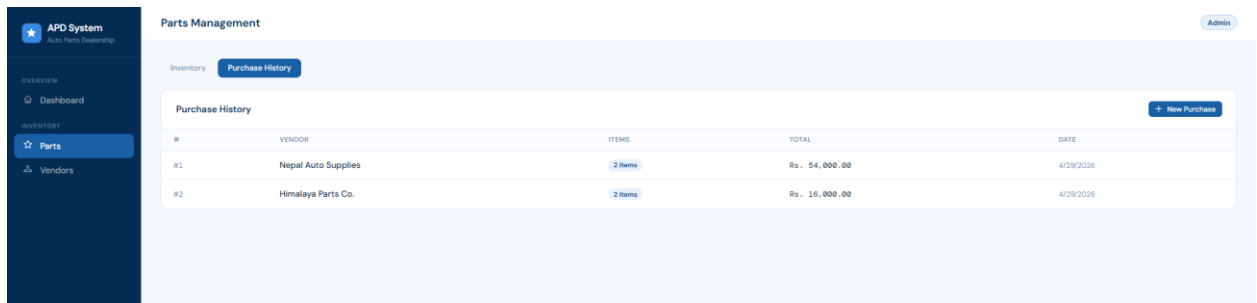
CreatePartDTO name* > [...] description > [...] unitPrice* > [...] stockQuantity* > [...]
CreatePurchaseDTO vendorId* > [...] items* > [...]
CreateVendorDTO name* > [...] phone* > [...] email > [...] address > [...]
PurchasePartItemDTO partId* > [...] quantity* > [...] unitCost* > [...]
UpdatePartDTO name* > [...] description > [...] unitPrice* > [...] stockQuantity* > [...]
UpdateVendorDTO name* > [...] phone* > [...] email > [...] address > [...]

Frontend:

NAME	PRICE	STOCK
Brake Pad	Rs. 1,200.00	45
Engine Oil Filter	Rs. 350.00	8
Spark Plug	Rs. 180.00	120
Air Filter	Rs. 450.00	5
Clutch Plate	Rs. 3,200.00	22
NAME	PHONE	ORDERS
Nepal Auto Supplies	9841123456	12
Himalaya Parts Co.	9851234567	7
Pokhara Motors Ltd.	9886543210	3

PART NAME	DESCRIPTION	UNIT PRICE	STOCK	ADDED	
Brake Pad	Front disc brake pad	Rs. 1,200.00	45	4/29/2026	 
Engine Oil Filter	Compatible with Honda & Toyota	Rs. 350.00	8 LOW	4/29/2026	 
Spark Plug	NGK standard	Rs. 180.00	120	4/29/2026	 
Air Filter	Universal fit	Rs. 450.00	5 LOW	4/29/2026	 
Clutch Plate	Heavy duty	Rs. 3,200.00	22	4/29/2026	 





3.2.3 Feature 5: Admin can manage vendor details (CRUD operations)

Backend:

Vendors		^
GET	/api/Vendors	▼
POST	/api/Vendors	▼
GET	/api/Vendors/{id}	▼ 
PUT	/api/Vendors/{id}	▼
DELETE	/api/Vendors/{id}	▼

- **GET /api/Vendors** - Fetches all vendors from the database. This is what loads Vendor Directory table.
- **POST /api/Vendors** - Creates a new vendor. Receives name, phone, email, and address and saves it to the database.
- **GET /api/Vendors/{id}** - Fetches a single vendor by ID including their total purchase count.
- **PUT /api/Vendors/{id}** - Updates an existing vendor's details by ID such as phone number or address.
- **DELETE /api/Vendors/{id}** - Deletes a vendor by ID. Returns a conflict error if the vendor has existing purchases linked to them.

GET/api/Vendors

Parameters

Cancel

No parameters

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'https://localhost:44310/api/Vendors' \
-H 'accept: */*'

```

Request URL

```
https://localhost:44310/api/Vendors

```

Server response

CodeDetails

200

Response body

[]Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 06:29:12 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET

```

Responses

CodeDescription

200OK

Links

Active

No links

Go to Se

QueryQuery History

1SELECT * FROM public."Vendors"

2ORDER BY "VendorId" ASC

Data OutputMessagesNotifications

VendorId

[PK] integer

Name

character varying (100)

Phone

character varying (15)

Email

text

Address

text

CreatedAt

timestamp with time zone

18

POST

/api/Vendors

Parameters

Cancel

Reset

No parameters

Request body

application/json

```
{
  "name": "Dishant",
  "phone": "9800000000",
  "email": "user@example.com",
  "address": "Butwal"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  https://localhost:44310/api/Vendors' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Dishant",
    "phone": "9800000000",
    "email": "user@example.com",
    "address": "Butwal"
  }'
```

Request URL

https://localhost:44310/api/Vendors

Server response

Code

Details

201

Undocumented

Response body

```
{
  "vendorId": 1,
  "name": "Dishant",
  "phone": "9800000000",
  "email": "user@example.com",
  "address": "Butwal",
  "createdAt": "2026-04-29T06:31:51.2123672Z",
  "totalPurchases": 0
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 06:31:51 GMT
location: https://localhost:44310/api/Vendors/1
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code

Description

Links

200

OK

No links

Figure 10: Add New Vendor

Query

Query History

1

SELECT * FROM public."Vendors"

2

ORDER BY "VendorId" ASC

Data Output

Messages

Notifications

Showing rows: 1 to 1

	VendorId [PK] integer	Name character varying (100)	Phone character varying (15)	Email text	Address text	CreatedAt timestamp with time zone
1	1	Dishant	9800000000	user@example.co...	Butwal	2026-04-29 12:16:51.212367+05:...

GET

/api/Vendors/{id}

Parameters

Try it out

Name

Description

id * required

integer(\$int32)

id

(path)

Responses

Code

Description

Links

200

OK

No links

PUT

/api/Vendors/{id}

Parameters

Try it out

Name

Description

id * required

integer(\$int32)

id

(path)

Request body

application/json

Example Value

Schema

{

"name": "string",

"phone": "string",

"email": "user@example.com",

"address": "string"

}

Responses

Code

Description

Links

200

OK

No links

Figure 11: Update Vendor Details

DELETE

/api/Vendors/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	1
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  https://localhost:44310/api/Vendors/1 \
  -H 'accept: */*'
```

Request URL

https://localhost:44310/api/Vendors/1

Server response

Code	Details
200	Response body <pre>{ "message": "Vendor deleted successfully." }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 29 Apr 2026 06:35:01 GMT server: Microsoft-IIS/10.0 x-powered-by: ASP.NET</pre>

Responses

Code	Description	Links
200	OK	No links available

Query

Query History

1

SELECT * FROM public."Vendors"

2

ORDER BY "VendorId" ASC

Data Output

Messages

Notifications

VendorId	Name	Phone	Email	Address	CreatedAt
[PK] integer	character varying (100)	character varying (15)	text	text	timestamp with time zone

Figure 12: Delete Vendor

3.2.4 Frontend:

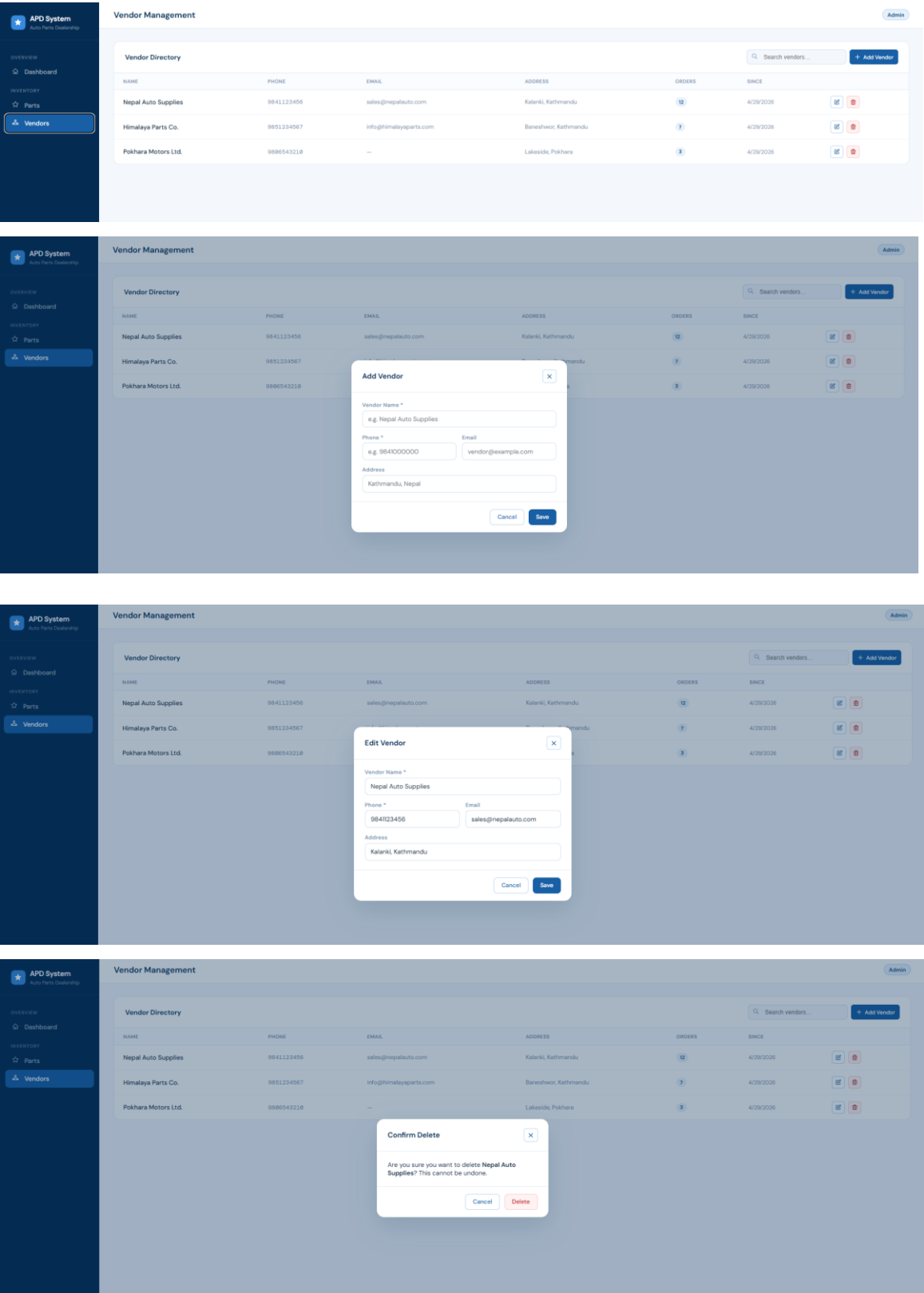


Figure 13: Frontend part of Vendor And Parts management

3.2.5 Member 3: Kushal Chandra Shrestha

Feature 13:

Appointment		^
POST	/api/Appointment	✓
GET	/api/Appointment	✓
GET	/api/Appointment/user/{userId}	✓

Figure 14: List of API for managing Appointment

- POST /api/appointment - Creates a new appointment for user, takes user id, vehicle id, appointment date time and defaults to pending status.
- GET /api/appointment- Fetches all the appointments for the system.
- GET /api/appointment/user/{userId}- Only fetches the appointment of a particular user.

Appointment

POST

/api/Appointment

Parameters

Cancel

Reset

No parameters

Request body

application/json

```
{
  "userId": 2,
  "vehicleId": 2,
  "appointmentDate": "2026-04-29",
  "appointmentTime": "10:15:00"
}
```

Execute

Clear

http://localhost:5090/api/Appointment

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "message": "Appointment created successfully.",
  "data": {
    "appointmentId": 1,
    "userId": 2,
    "userName": "Kushal",
    "vehicleId": 2,
    "vehicleNumber": "Ba. 14 6198",
    "appointmentDateTime": "2026-04-29T00:00:00Z",
    "status": "Pending",
    "createdAt": "2026-04-29T00:09:32.8281397Z"
  },
  "errors": null
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 00:09:32 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Figure 15: Create Appointment Request

Query

Query History

```
1 SELECT * FROM public."Appointments"
2 ORDER BY "AppointmentId" ASC
```

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1

	AppointmentId [PK] integer	UserId integer	VehicleId integer	AppointmentDateTime timestamp with time zone	Status text	CreatedAt timestamp with time zone
1	1	2	2	2026-04-29 10:15:00+05:45	completed	2026-04-29 00:09:32.828139+05:45

Figure 16: Appointment Saved in Database

GET /api/Appointment

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5090/api/Appointment' \
  -H 'accept: */*'
```

Request URL

http://localhost:5090/api/Appointment

Server response

Code	Details
200	Response body <pre>{ "success": true, "message": "Appointments retrieved successfully.", "data": [{ "appointmentId": 1, "userId": 2, "userName": "Kushal", "vehicleId": 2, "vehicleNumber": "Ba. 14 6198", "appointmentDateTime": "2026-04-29T00:00+05:45", "status": "Pending", "createdAt": "2026-04-29T00:09:32.828139+05:45" }], "errors": null }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 29 Apr 2026 00:11:02 GMT server: Kestrel transfer-encoding: chunked</pre>

Figure 17: Get All Appointments (GET /api/appointment)

GET /api/Appointment/user/{userId}

Parameters Cancel

Name	Description
userId * required	
integer(\$int32)	2
(path)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5090/api/Appointment/user/2' \
  -H 'accept: */*'
```

Request URL

http://localhost:5090/api/Appointment/user/2

Figure 18: Fetch User Appointments by searching with userId (GET /api/appointment/user/{userId})



Figure 19: Successfully Retrieved Appointment by userId

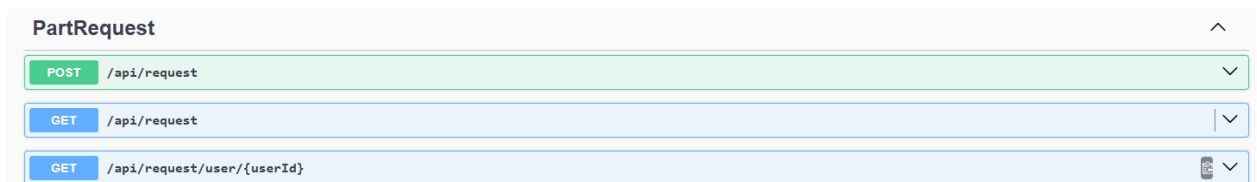


Figure 20: Part request API

- POST /api/request - Creates a part request for unavailable parts.
- GET /api/request - Gets all the part requests in the system.
- GET /api/request/user/{userId}- Gets the part requests for a particular user.

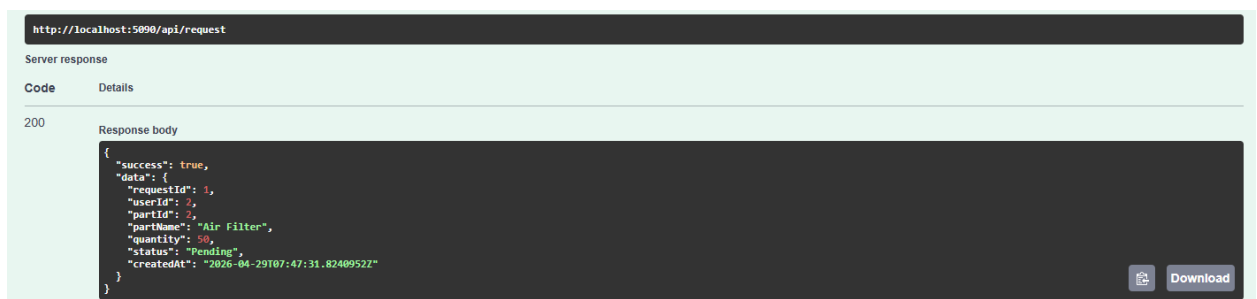
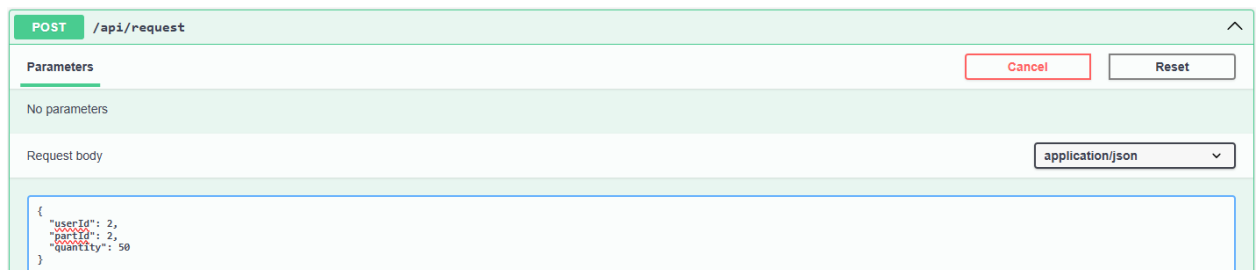


Figure 21: Creates a part request for unavailable parts.

Query

Query History

1

SELECT * from "PartRequests"

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing row

	RequestId [PK] integer	UserId integer	PartId integer	Quantity integer	Status text	CreatedAt timestamp without time zone
1	1	2	2	50	Pending	2026-04-29 07:47:31.824095

GET

/api/request

⌵

Parameters

No parameters

Cancel

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5090/api/request' \
  -H 'accept: */*'
```

Request URL

http://localhost:5090/api/request

Request URL

http://localhost:5090/api/request

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "data": [
    {
      "requestId": 2,
      "userId": 3,
      "partId": 4,
      "partName": "Spark Plug (Set of 4)",
      "quantity": 50,
      "status": "Pending",
      "createdAt": "2026-04-29T07:48:41.569827Z"
    },
    {
      "requestId": 1,
      "userId": 2,
      "partId": 2,
      "partName": "Air Filter",
      "quantity": 50,
      "status": "Pending",
      "createdAt": "2026-04-29T07:47:31.824095Z"
    }
  ]
}
```

📄

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 07:48:48 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	Success	No links

Figure 22: Get All Part Requests

27

GET /api/request/user/{userId}

Parameters

Name	Description
userId * required	
integer (int32)	2
(path)	

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5090/api/request/user/2' \
-H 'accept: */*'
```

Request URL

```
http://localhost:5090/api/request/user/2
```

Server response

Code **Details**

200

Response body

```
{
  "success": true,
  "data": [
    {
      "requestId": 1,
      "userId": 2,
      "partId": 2,
      "partName": "Air Filter",
      "quantity": 50,
      "status": "Pending",
      "createdAt": "2026-04-29T07:47:31.824095Z"
    }
  ]
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 07:49:46 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	Success	No links

Figure 23: Get User Part Requests

Review

- POST** /api/review
- GET** /api/review
- GET** /api/review/user/{userId}

Figure 24: List of Api to POST and GET the review

- **POST /api/review**- Creates a review for an appointment for user.
- **GET /api/review**- Fetch all the reviews in the system.
- **GET /api/reviews/user/{userId}**- Fetches only the reviews of one user .

POST

/api/review

Parameters

No parameters

Request body

application/json

```
{
  "userId": 2,
  "appointmentId": 1,
  "rating": 4,
  "comment": "Good service."
}
```

http://localhost:5090/api/review

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "data": {
    "reviewId": 1,
    "userId": 2,
    "userName": "Kushal",
    "appointmentId": 1,
    "appointmentLabel": "Service - 29 Apr 2026",
    "rating": 4,
    "comment": "Good service.",
    "createdAt": "2026-04-29T08:36:37.254945Z"
  }
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 08:36:36 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Query

Query History

1

SELECT * FROM public."Reviews"

2

ORDER BY "ReviewId" ASC

Data Output

Messages

Notifications

Showing rows: 1 to

	ReviewId [PK] integer	UserId integer	Rating integer	Comment text	CreatedAt timestamp with time zone	AppointmentId integer
1	1	2	4	Good service.	2026-04-29 08:36:37.254945+05:45	1

GET

/api/review

Cancel

Parameters

No parameters

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5090/api/review' \
  -H 'accept: */*'

```

Request URL

http://localhost:5090/api/review

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "data": [
    {
      "reviewId": 1,
      "userId": 2,
      "userName": "Kushal",
      "appointmentId": 1,
      "appointmentLabel": "Service - 29 Apr 2026",
      "rating": 4,
      "comment": "Good service.",
      "createdAt": "2026-04-29T08:36:37.254945+05:45"
    }
  ]
}
```

Download

GET

/api/review/user/{userId}

Cancel

Parameters

Name	Description
userId * required	
integer(\$int32)	2
(path)	

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5090/api/review/user/2' \
  -H 'accept: */*'

```

Request URL

http://localhost:5090/api/review/user/2

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "data": [
    {
      "reviewId": 1,
      "userId": 2,
      "userName": "Kushal",
      "appointmentId": 1,
      "appointmentLabel": "Service - 29 Apr 2026",
      "rating": 4,
      "comment": "Good service.",
      "createdAt": "2026-04-29T08:36:37.254945+05:45"
    }
  ]
}
```

Download

Response headers

3.2.6 Member 4: Pratik Prasai

Feature 1: Admin can generate and view financial reports (daily, monthly, yearly)

Frontend:

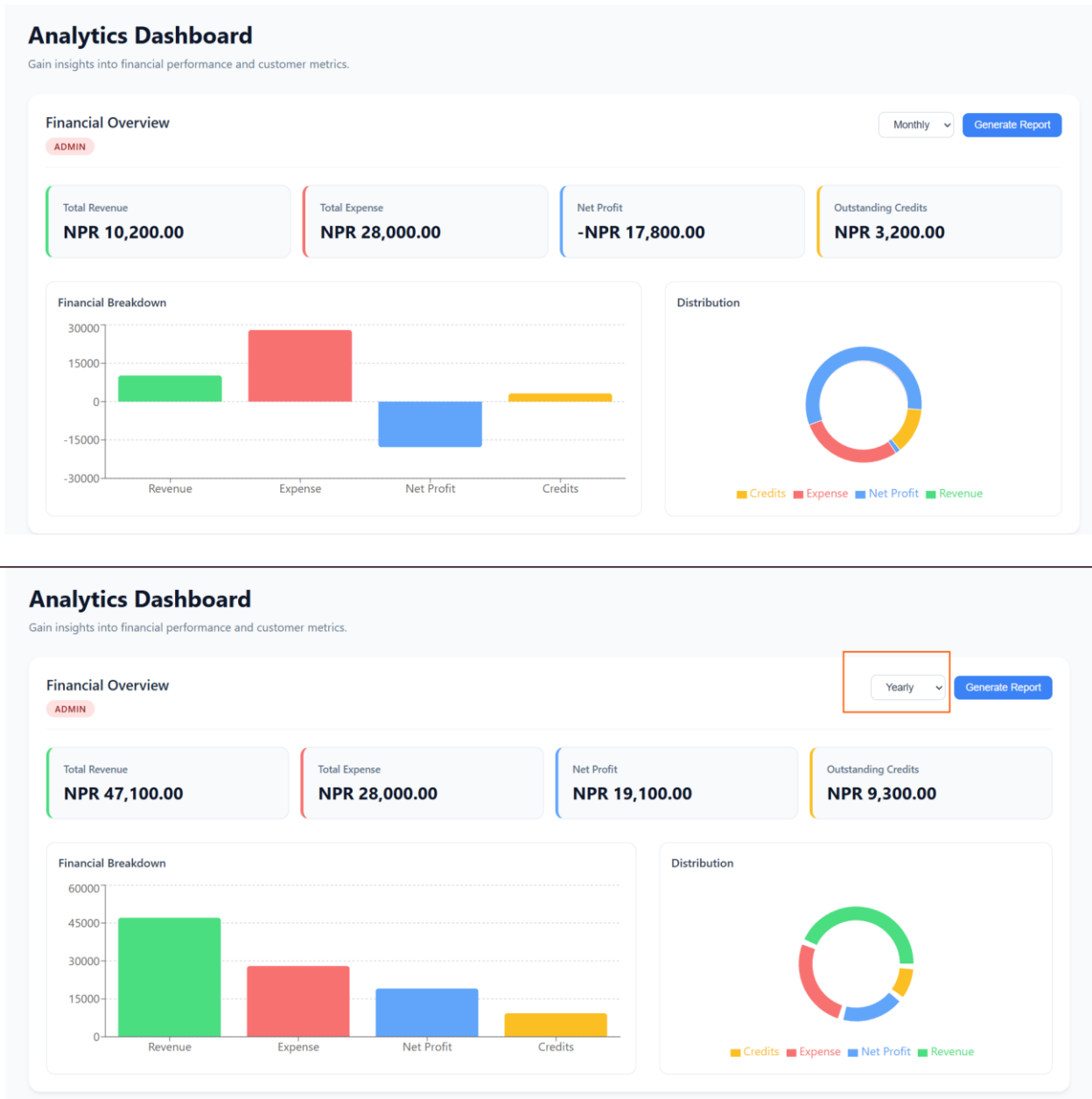
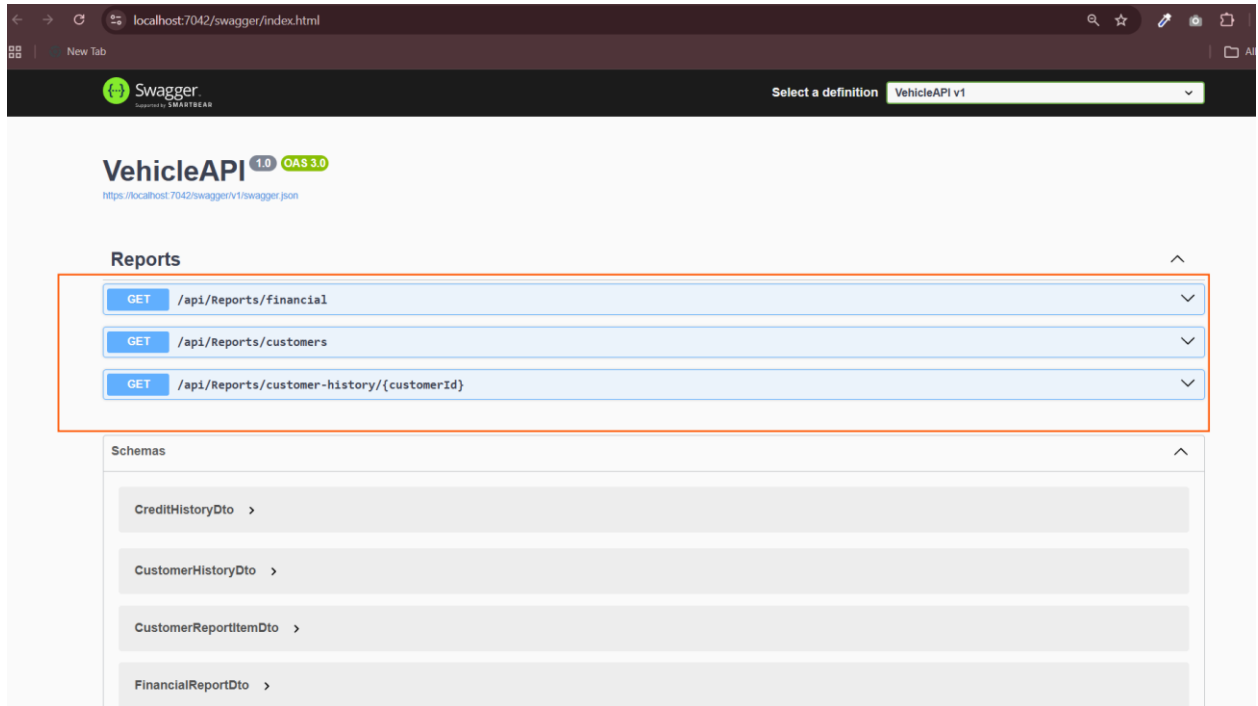
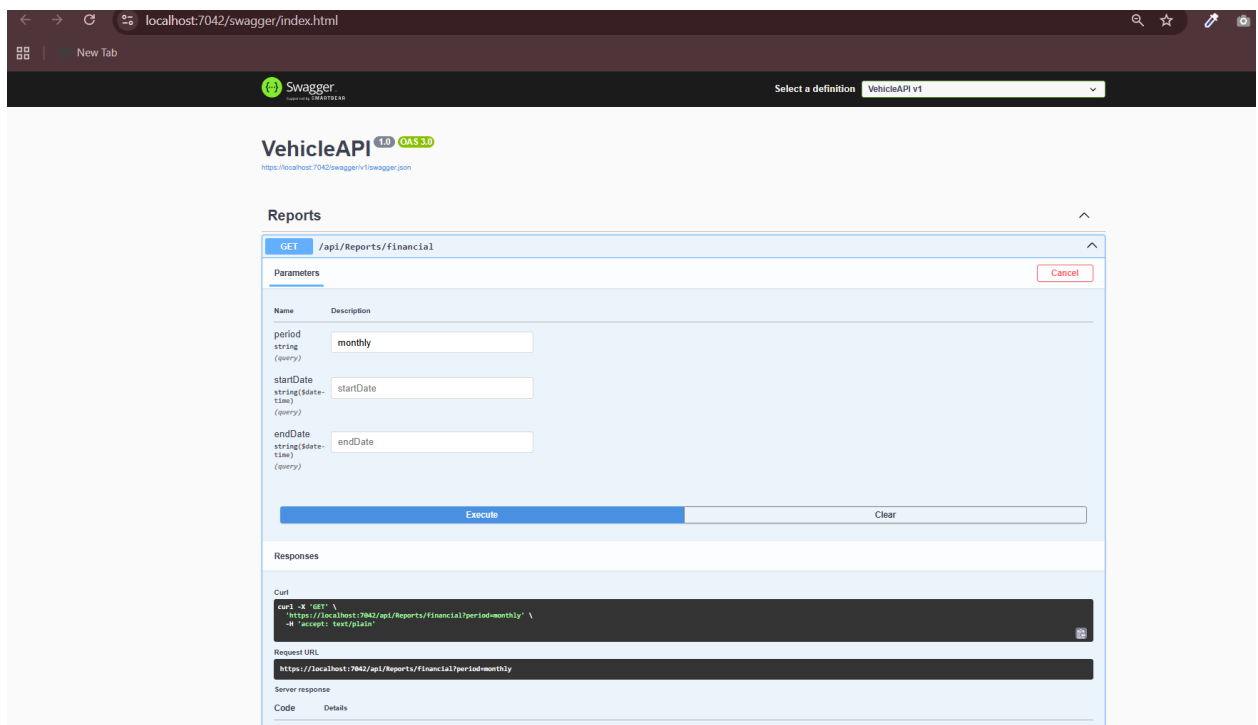


Figure 25: Yearly Financial Report

Backend:



The image shows the Swagger UI for the VehicleAPI v1. The browser address bar displays 'localhost:7042/swagger/index.html'. The Swagger logo is in the top left, and a dropdown menu in the top right shows 'Select a definition' with 'VehicleAPI v1' selected. The main heading is 'VehicleAPI 1.0 OAS 3.0' with the URL 'https://localhost:7042/swagger/v1/swagger.json'. Below this, the 'Reports' section is expanded, showing three endpoints: 'GET /api/Reports/financial', 'GET /api/Reports/customers', and 'GET /api/Reports/customer-history/{customerId}'. Each endpoint has a dropdown arrow. Below the Reports section, the 'Schemas' section is also expanded, showing four schemas: 'CreditHistoryDto', 'CustomerHistoryDto', 'CustomerReportItemDto', and 'FinancialReportDto', each with a right-pointing arrow.



The image shows the Swagger UI for the VehicleAPI v1, specifically the details for the 'GET /api/Reports/financial' endpoint. The browser address bar displays 'localhost:7042/swagger/index.html'. The Swagger logo is in the top left, and a dropdown menu in the top right shows 'Select a definition' with 'VehicleAPI v1' selected. The main heading is 'VehicleAPI 1.0 OAS 3.0' with the URL 'https://localhost:7042/swagger/v1/swagger.json'. Below this, the 'Reports' section is expanded, and the 'GET /api/Reports/financial' endpoint is selected. The endpoint details are shown in a light blue box. The 'Parameters' section is expanded, showing three query parameters: 'period' (string, 'monthly'), 'startDate' (string, 'startDate'), and 'endDate' (string, 'endDate'). Below the parameters, there are 'Execute' and 'Clear' buttons. The 'Responses' section is also expanded, showing a 'Curl' command: 'curl -X GET https://localhost:7042/api/Reports/financial/period-monthly -H "accept: text/plain"'. Below the curl command, there is a 'Request URL' field with the value 'https://localhost:7042/api/Reports/financial/period-monthly'. Below the request URL, there is a 'Server response' section with a table showing 'Code' and 'Details'. The table has one row with the value '200' in the 'Code' column and 'OK' in the 'Details' column.

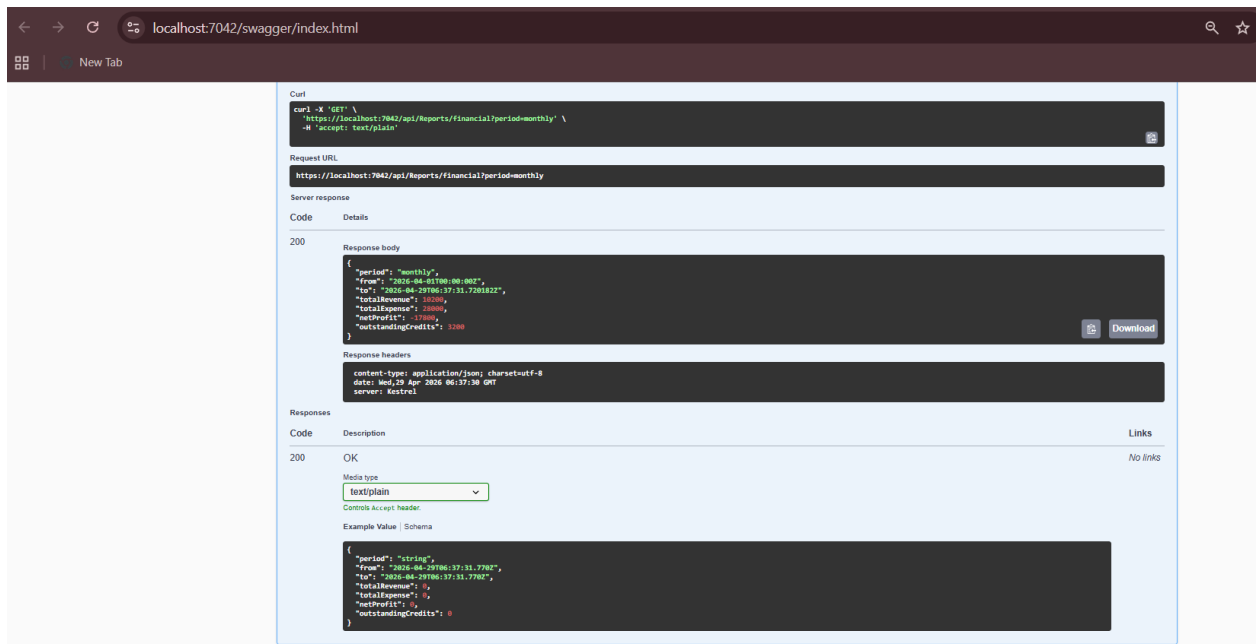
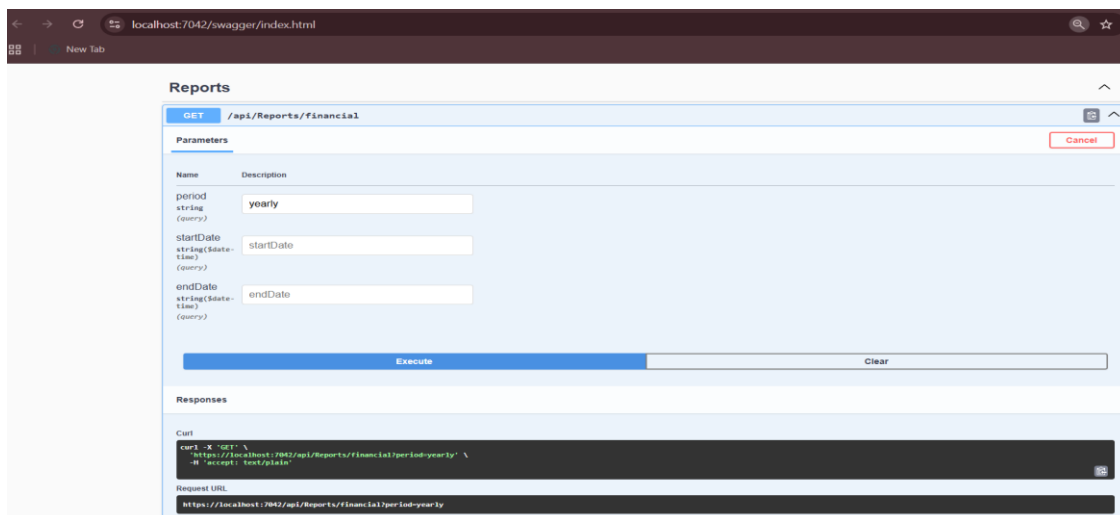


Figure 26: Api to fetch monthly financial report



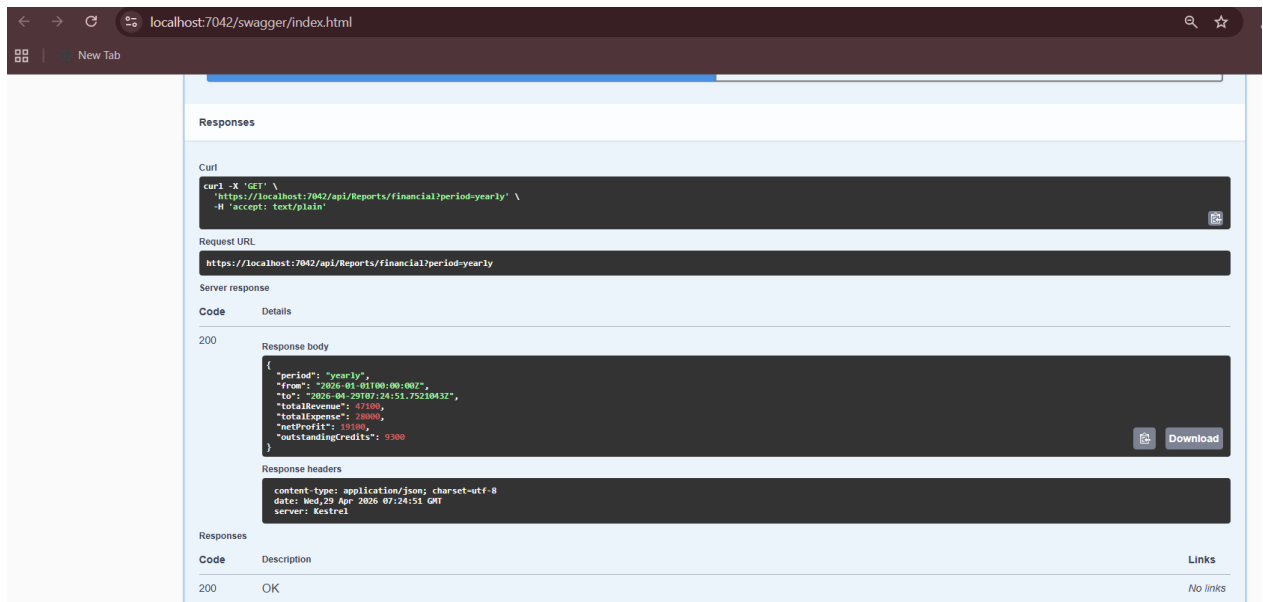
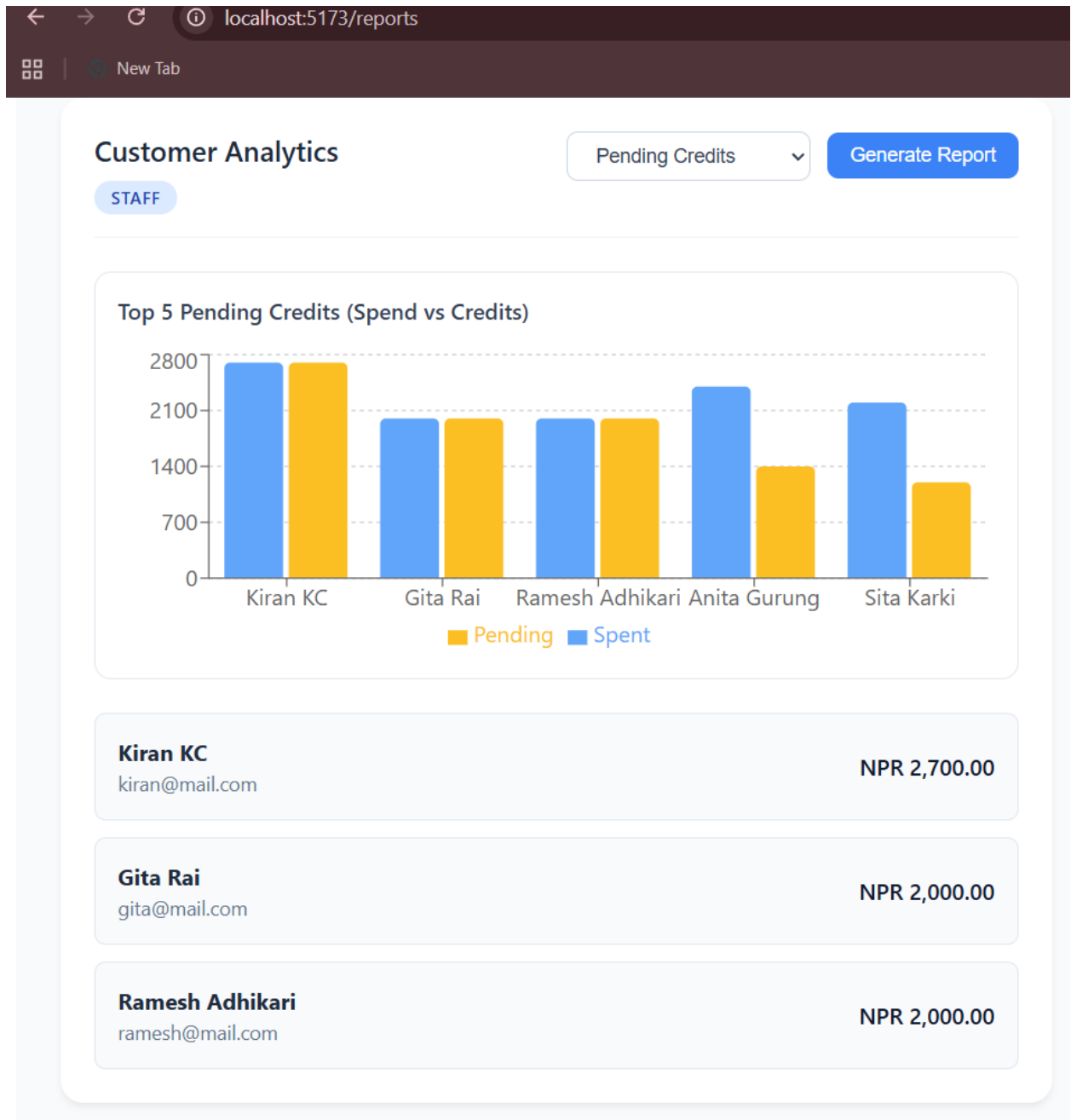


Figure 27: Financial Report Yearly

GET /api/reports/financial returns a financial analytics summary for a selected time range. It totals revenue by summing Sales.FinalAmount, totals expense by summing Purchases.TotalAmount, calculates net profit as revenue minus expense, and also totals outstanding unpaid credits by summing Credits.AmountDue where IsPaid is false. The time window is controlled by the period query parameter (daily, monthly default, or yearly), and you can override that by providing startDate and endDate (both optional).

Feature 9: Staff can generate customer-related reports (regulars, high spenders, pending credits)

Frontend:



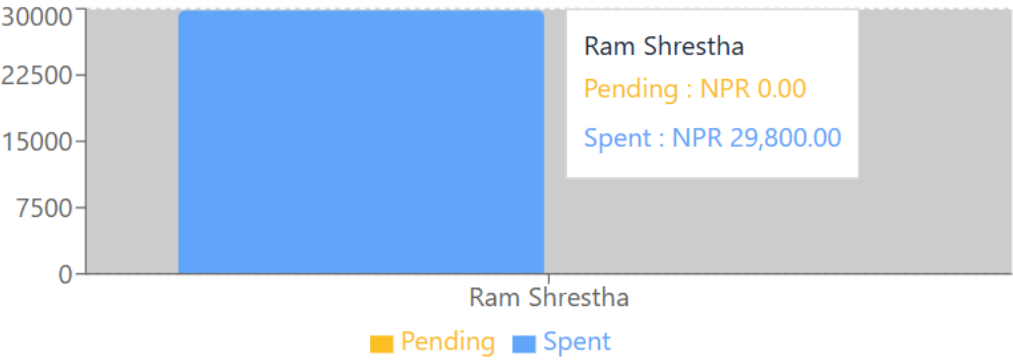
Customer Analytics

High Spenders

Generate Report

STAFF

Top 5 High Spenders (Spend vs Credits)



Ram Shrestha
ram@mail.com

NPR 29,800.00

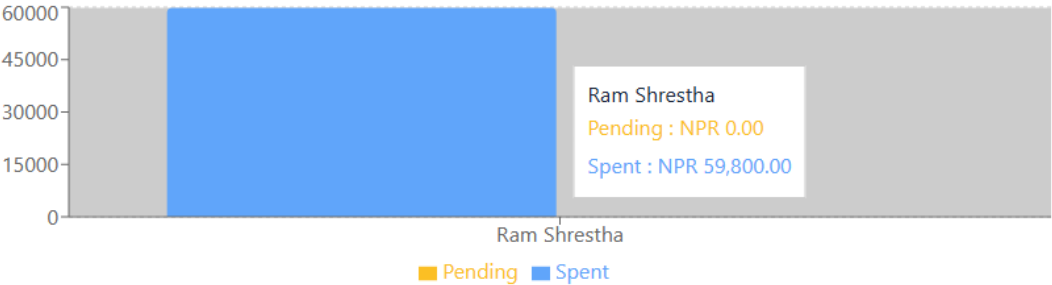
Customer Analytics

Regular Customers

Generate Report

STAFF

Top 5 Regular Customers (Spend vs Credits)



Ram Shrestha
ram@mail.com

NPR 59,800.00

Backend:

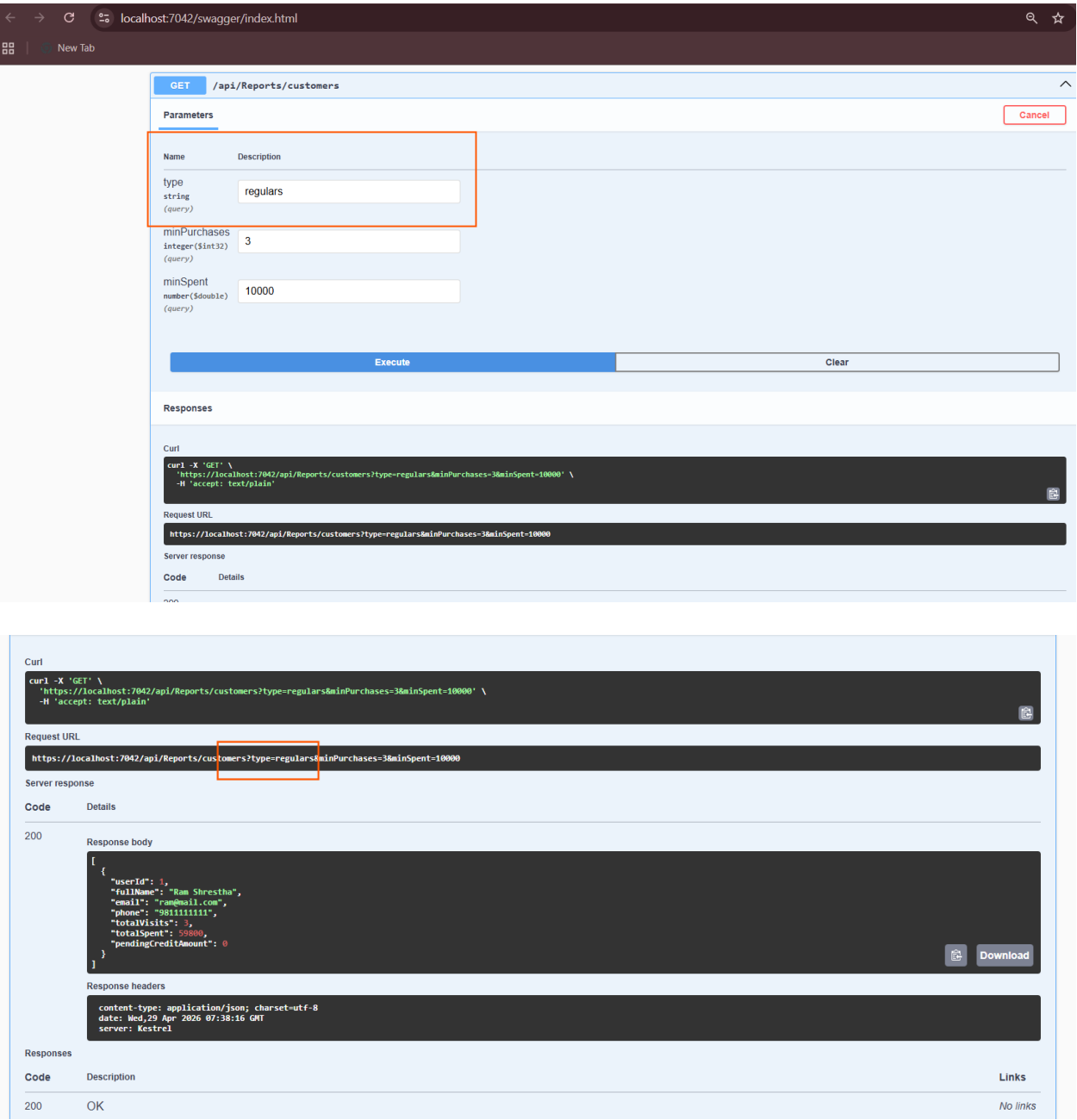


Figure 28: Regulars Report Response

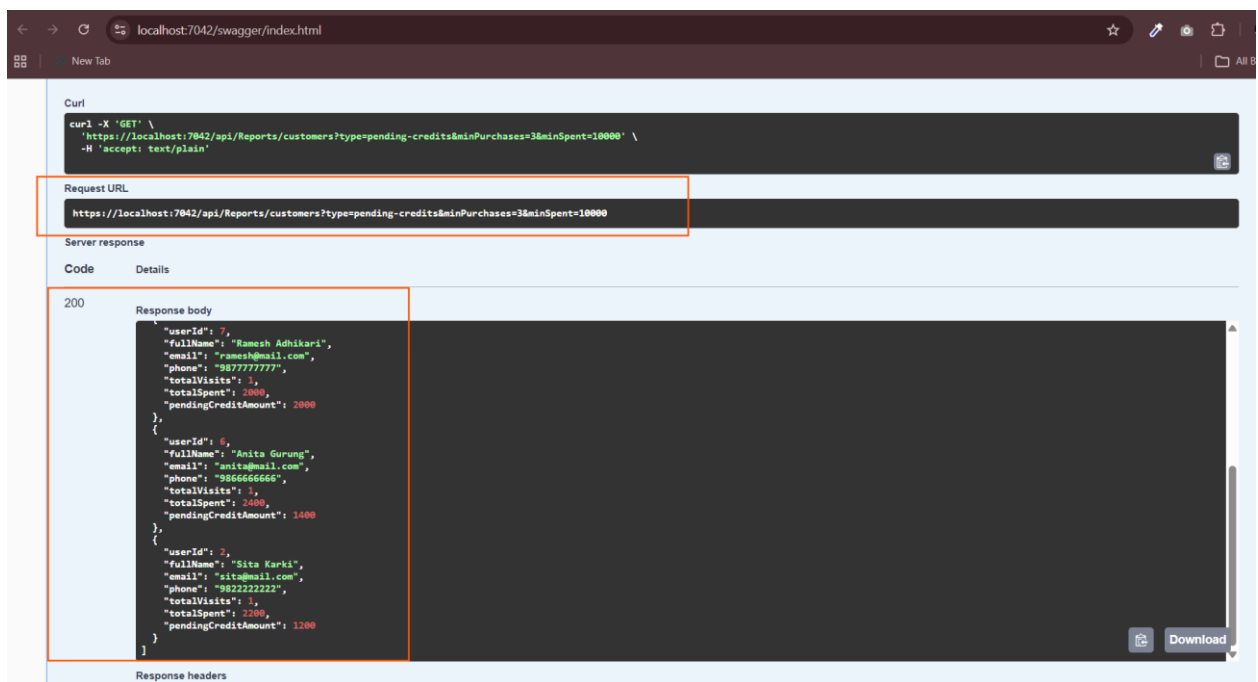
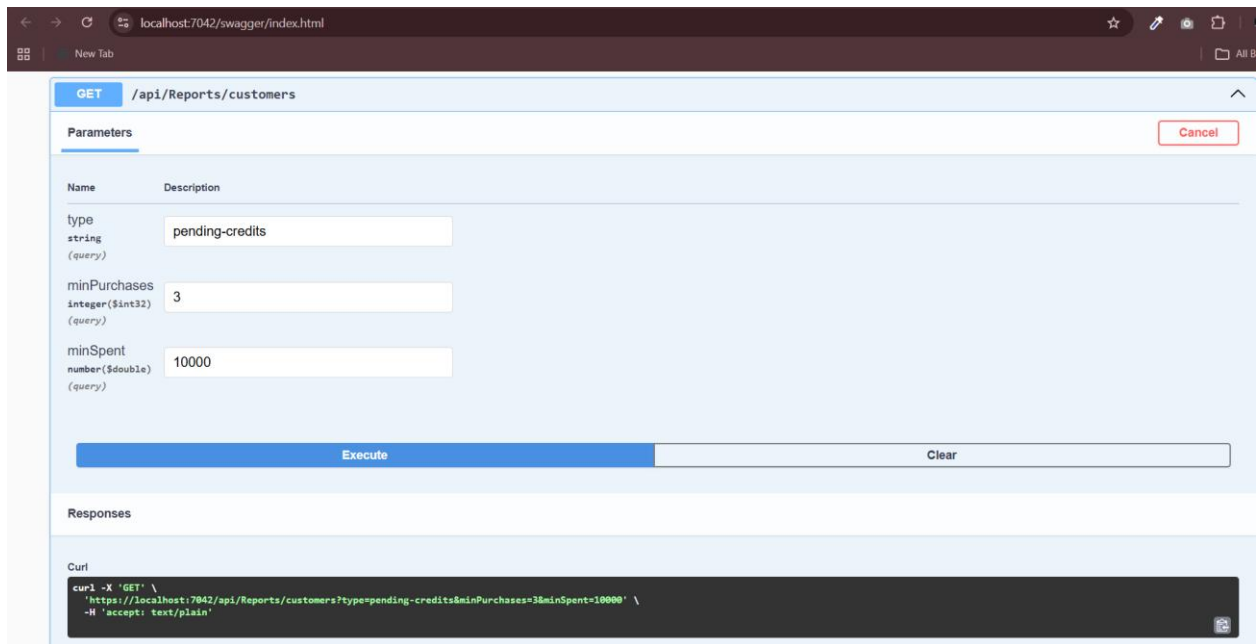


Figure 29: Pending Credits Report Response

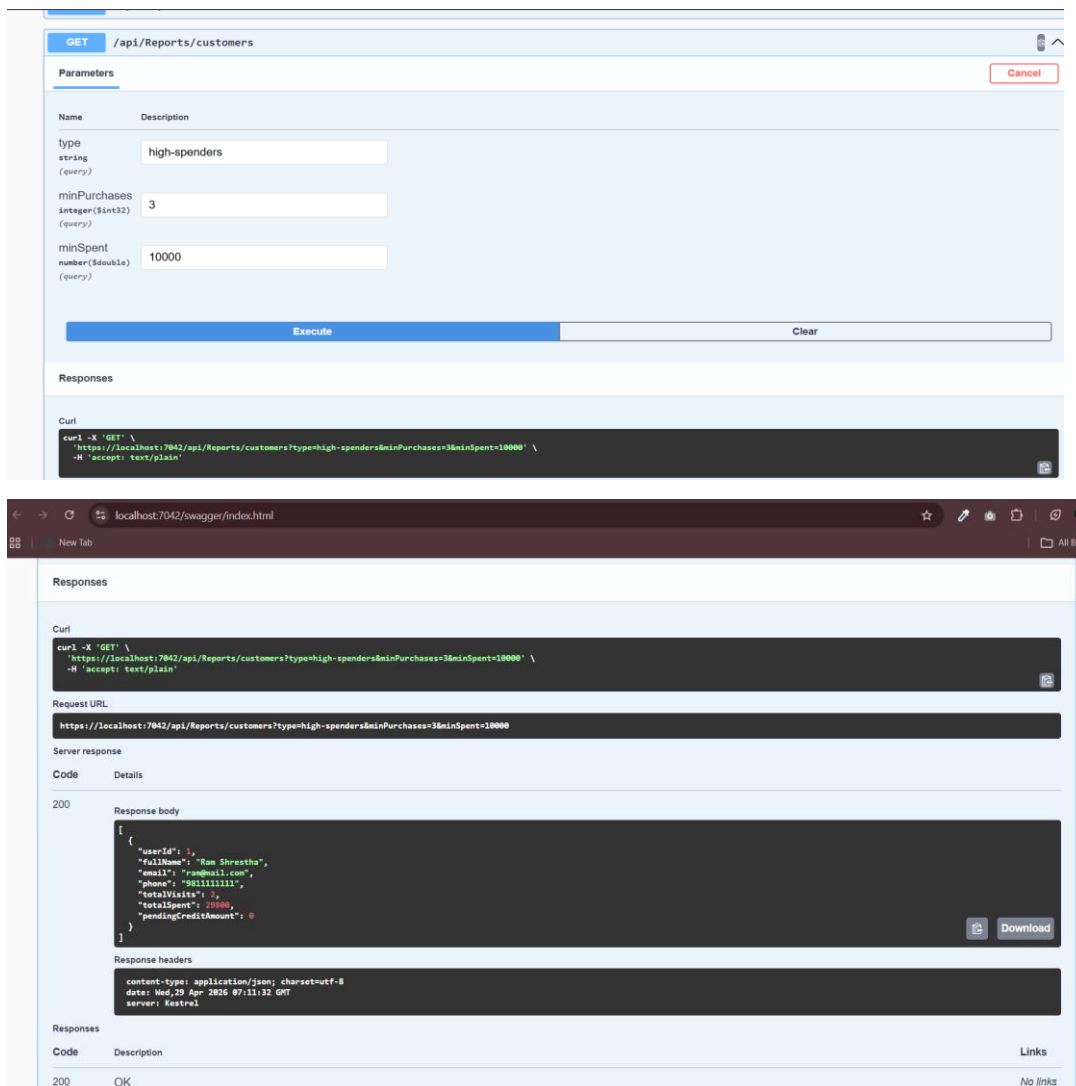


Figure 30: High spender's Report Response

The `/api/reports/customers` API provides a simple report about customers (specifically users with `RoleId = 3`). For each customer, it calculates how many times they have made purchases (total visits), how much money they have spent in total (total spent), and how much money they still owe (pending credit). The API also allows filtering results using a type parameter. For example, you can get regular customers who have made a minimum number of purchases (default is 3), high-spending customers who have spent above a certain amount (default is 10,000), or customers who still have unpaid credit. In the end, it returns a list of customers along with these calculated details.

3.2.7 Member 5: Slesha Rawal

Feature 7: Staff can sell vehicle parts and create sales invoices

Backend:

Sale		^
GET	/api/sale	▼
POST	/api/sale	▼
GET	/api/sale/{id}	▼
GET	/api/sale/user/{userId}	▼

- GET /api/sale: Fetches the whole list of Sales records from the whole system.
- POST /api/sale: Creates the whole new sale order.
- GET /api/sale/{id}: Fetches the receipt/invoice details of individual sale using Sale ID.
- GET /api/sale/user/{userId}: Fetches the history of sale of individual customers.

GET /api/sale

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5090/api/sale' \
  -H 'accept: */*'
```

Request URL

http://localhost:5090/api/sale

Figure 31: Fetch all the available sales

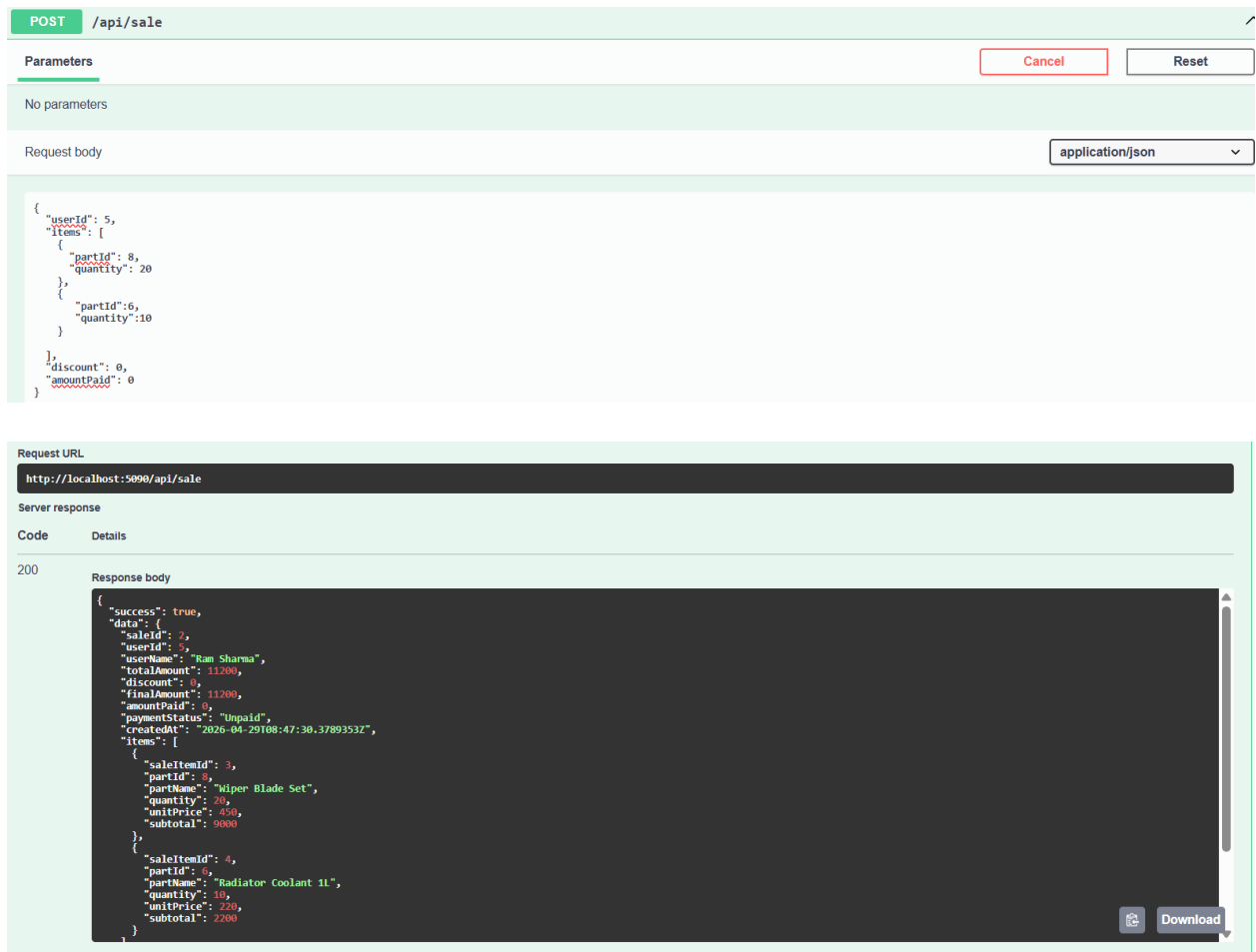


Figure 32: Create Sale Request

GET

/api/sale/{id}

^

Parameters

Cancel

Name	Description
id * required integer(\$int32) (path)	1

Execute

Clear

Responses

server response

Code

Details

200

Response body

```

{
  "success": true,
  "data": {
    "saleId": 1,
    "userId": 2,
    "userName": "Kushal",
    "totalAmount": 26200,
    "discount": 0,
    "finalAmount": 26200,
    "amountPaid": 0,
    "paymentStatus": "Unpaid",
    "createdAt": "2026-04-29T08:46:53.255311+05:45",
    "items": [
      {
        "saleItemId": 1,
        "partId": 5,
        "partName": "Timing Belt",
        "quantity": 20,
        "unitPrice": 1200,
        "subTotal": 24000
      },
      {
        "saleItemId": 2,
        "partId": 6,
        "partName": "Radiator Coolant 1L",
        "quantity": 10,
        "unitPrice": 220,
        "subTotal": 2200
      }
    ]
  }
}

```

Download

Response headers

```

content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 08:49:06 GMT
server: Kestrel
transfer-encoding: chunked

```

Responses

Code	Description

GET

/api/sale/user/{userId}

^

Parameters

Cancel

Name	Description
userId * required integer(\$int32) (path)	5

Execute

Clear

Code

Details

200

Response body

```
{
  "success": true,
  "data": [
    {
      "saleId": 2,
      "userId": 5,
      "userName": "Ram Sharma",
      "totalAmount": 11200,
      "discount": 0,
      "finalAmount": 11200,
      "amountPaid": 0,
      "paymentStatus": "Unpaid",
      "createdAt": "2026-04-29T08:47:30.378935+05:45",
      "items": [
        {
          "saleItemId": 3,
          "partId": 4,
          "partName": "Wiper Blade Set",
          "quantity": 20,
          "unitPrice": 450,
          "subtotal": 9000
        },
        {
          "saleItemId": 4,
          "partId": 6,
          "partName": "Radiator Coolant 1L",
          "quantity": 10,
          "unitPrice": 220,
          "subtotal": 2200
        }
      ]
    }
  ]
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 29 Apr 2026 08:58:08 GMT
server: Kestrel
```

Figure 33: Generated invoice of sales

Feature 8: Staff can view customer details, history, and vehicle info

Backend:

GET

/api/Customer/{id}/history

Cancel

Parameters

Name	Description
id required	
integer(int32)	5
(path)	

ExecuteClear

Responses

- GET /api/customer/{id}/history- fetches the customer history like appointment , service and vehicle details.

Request URL

http://localhost:5090/api/Customer/5/history

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "data": {
    "customer": {
      "userId": 5,
      "fullName": "Ram Sharma",
      "email": "ram@email.com",
      "phone": "9800000003",
      "address": null,
      "isActive": true,
      "createdAt": "2026-04-29T13:29:19.708113+05:45",
      "vehicleNumber": "BA 1 KA 1234",
      "brand": "Toyota",
      "model": "Corolla",
      "year": 2018
    },
    "vehicles": [
      {
        "vehicleId": 3,
        "vehicleNumber": "BA 1 KA 1234",
        "brand": "Toyota",
        "model": "Corolla",
        "year": 2018
      }
    ],
    "appointments": [],
    "purchases": [
      {
        "saleId": 2,
        "itemId": 3,
        "partId": 0,
        "partName": "Wiper Blade Set",
        "quantity": 20,
        "unitPrice": 450,
        "subTotal": 9000
      },
      {
        "saleId": 4,
        "partId": 0,
        "partName": "Radiator Coolant 1L",
        "quantity": 10,
        "unitPrice": 220,
        "subTotal": 2200
      }
    ],
    "credit": {
      "creditId": 2,
      "amount": 11200,
      "paymentStatus": "Unpaid",
      "createdAt": "2026-04-29T08:47:30.378935+05:45"
    }
  }
}
```

Download

Response headers

200

Response body

```
{
  "saleId": 2,
  "userId": 5,
  "userName": "Ram Sharma",
  "totalAmount": 11200,
  "discount": 0,
  "finalAmount": 11200,
  "amountPaid": 0,
  "paymentStatus": "Unpaid",
  "createdAt": "2026-04-29T08:47:30.378935+05:45",
  "items": [
    {
      "saleItemId": 3,
      "partId": 0,
      "partName": "Wiper Blade Set",
      "quantity": 20,
      "unitPrice": 450,
      "subTotal": 9000
    },
    {
      "saleItemId": 4,
      "partId": 0,
      "partName": "Radiator Coolant 1L",
      "quantity": 10,
      "unitPrice": 220,
      "subTotal": 2200
    }
  ],
  "credit": {
    "creditId": 2,
    "amount": 11200,
    "paymentStatus": "Unpaid",
    "createdAt": "2026-04-29T08:47:30.378935+05:45"
  }
}
```

Download

Response headers

Figure 34: Customer Purchase History Retrieval

4 Proof of Work

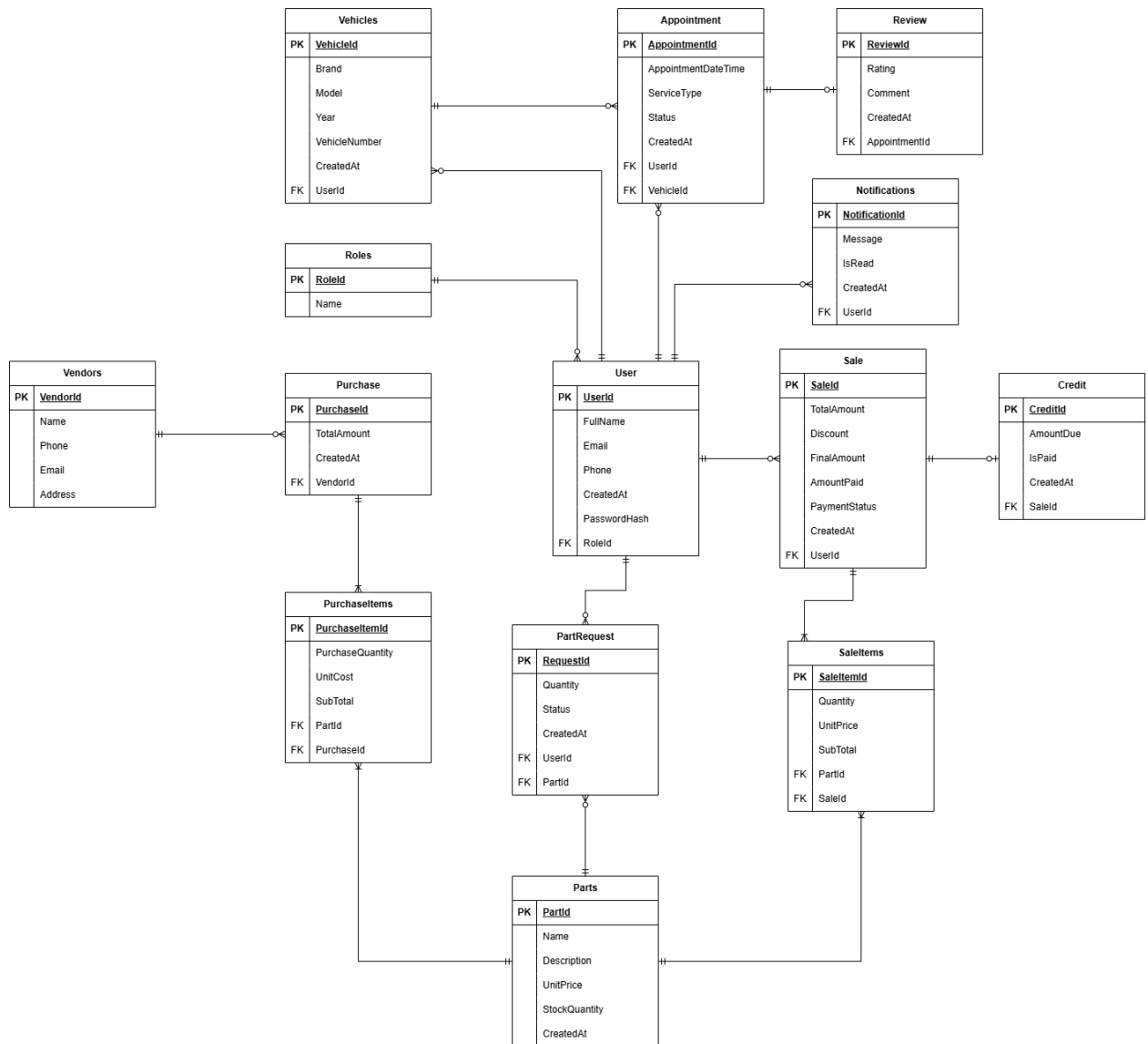


Figure 35: ER Diagram Representing Tables and Their Relationships

5 Individual reflection

5.1 Member 1: Bipin Kumar Thapa

I focused on customer and admin management. So far, all the customer registration and its associated vehicles, customer searching (by vehicle number, phone, ID, name) and staff registration and handling of their roles (admin panel planning/structure) are completed. In this regard, the task has enhanced the management of customer and staff information in the system.

Some pending tasks are admin staff management (fully handling registration and their roles in admin panel) and customer self registration with their profile and vehicle. When the task is completed, it will complete customer and staff data handling.

5.2 Member 2: Dishant Panjiyar

My responsibility was on managing inventory and supplier. He has already completed the management of parts (purchasing, editing, deleting parts) and suppliers (CRUD operation). He has done this by managing inventory and suppliers and so this functionality has been handled.

His remaining tasks is to manage admin purchase invoice creating process (so that it automatically updates inventory). When he completes this functionality, the inventory data management and tracking will be greatly eased.

5.3 Member 3: Kushal Chandra Shrestha

I covered the functionality that includes all those which interact with the customers (for automated processes). Customer appointment booking, service requests when any part is not available, service review, loyalty program (giving 10% discount if purchase amount is > 5000) has already been developed. Through this functionality, he has established the user's access and interaction to different systems.

The pending tasks are automatic system notification system (low stock alerts for admin, credit reminders to customers having unpaid credit for more than 1 month).

5.4 Member 4 : Pratik Prasai

I covered the reporting system. Financial reports (daily, monthly, yearly) and customer reports (regular customer, high spenders, pending credits) are already completed. These reports have been greatly useful for the management system.

His pending tasks include showing customer purchase history and service history. Through this, customer has his information in his account and admin also know what the customer purchases and which services he got.

5.5 Member 5 : Slesha Rawal

I covered the system of sales and giving details of the customer to staff. The selling process of parts and its Invoice generation and customer details and its history, vehicles can be accessed is already working well. This helps the admin and staff in day to day operations.

The pending tasks are customer email invoicing system. This task will complete the entire process of selling parts.